

Variational Multi-Task Learning with Gumbel-Softmax Priors

Anastasia Voznyuk

BMM 2025-2025

2025

Introduction

Traditional MTL assumes plenty of data and often only shares representations, not classifiers, or vice versa.

What if our tasks share only the same output space (e.g., same labels), but we have very little data per task and only a handful of related tasks are available to gain shared knowledge from.

We can leverage the **inductive bias** provided by related tasks learned **simultaneously** to better capture uncertainty caused by limited data.

- Consider a set of T related tasks, each of which is a classification problem. Tasks share the same label space for classification but have different distributions.
- Each task $t \in T$ has its own training data $\mathcal{D}_t = \{\mathbf{x}_{t,n}, \mathbf{y}_{t,n}\}_{n=1}^{N_t}$.
- Conditional predictive distribution for each task is denoted as $p(\mathcal{Y}_t | \mathcal{X}_t, \mathcal{D}_{T \setminus t})$.
- Our final goal is to maximize the conditional predictive log-likelihood:

$$\frac{1}{T} \sum_{t=1}^T \log p(\mathcal{Y}_t | \mathcal{X}_t, \mathcal{D}_{T \setminus t}) = \frac{1}{T} \sum_{t=1}^T \sum_{n=1}^{N_t} \log p(\mathbf{y}_{t,n} | \mathbf{x}_{t,n}, \mathcal{D}_{T \setminus t})$$

- We would like to incorporate **shared knowledge from related tasks** into the learning of both the classifier and the feature representation in a unified way by specifying their priors.

Latent Variables

We introduce the latent variables $\mathbf{w}_t$ and $\mathbf{z}_{t,n}$ to represent the classifier and the latent representation of the sample $\mathbf{x}_{t,n}$, respectively. The joint distribution for the task t can be factorized as

$$p(\mathcal{Y}_t, \mathcal{Z}_t, \mathbf{w}_t \mid \mathcal{X}_t, \mathcal{D}_{T \setminus t}) = \prod_{n=1}^{N_t} p(\mathbf{y}_{t,n} \mid \mathbf{z}_{t,n}, \mathbf{w}_t) p(\mathbf{z}_{t,n} \mid \mathbf{x}_{t,n}, \mathcal{D}_{T \setminus t}) p(\mathbf{w}_t \mid \mathcal{D}_{T \setminus t})$$

Note

The probabilistic latent variables capture uncertainties at different levels: $\mathbf{z}_{t,n}$ captures the uncertainty of each sample, while $\mathbf{w}_t$ works at the **categorical level**.

Overview of an idea

- Use variational Bayesian inference approach
- Add conditional priors over the latent variables for each task as a **mixture of the variational posteriors** of other tasks.

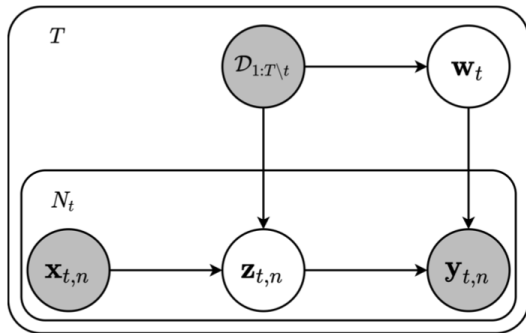


Figure: A graphical illustration of the proposed model for multi-task learning. $(\mathbf{x}_{t,n}, \mathbf{y}_{t,n})$ is the n -th sample from task t . $\mathbf{w}_t$ and $\mathbf{z}_{t,n}$ are the introduced latent variables, i.e., a classifier/regressor and representation, respectively, whose priors are conditioned on the data from other tasks,

Variational Bayesian Inference

Once again, we cannot obtain true joint posterior $p(\mathcal{Z}_t, \mathbf{w}_t | \mathcal{D})$, so we introduce a variational joint distribution $q(\mathcal{Z}_t, \mathbf{w}_t | \mathcal{D})$, for the current task to approximate the true posterior

$$q(\mathcal{Z}_t, \mathbf{w}_t | \mathcal{D}_t) = q_\theta(\mathbf{w}_t | \mathcal{D}_t) \prod_{n=1}^{N_t} q_\phi(\mathbf{z}_{t,n} | \mathbf{x}_{t,n})$$

where $q_\theta(\mathbf{w}_t | \mathcal{D}_t)$ and $q_\phi(\mathbf{z}_{t,n} | \mathbf{x}_{t,n})$ are variational posterior distributions for classifiers and latent representations respectively.

After that we use the classical pipeline of variational Bayesian inference using the ELBO trick and minimising the KL-divergence between q and p .

Getting Back to Priors

The aim is to learn which tasks are related so that each task can borrow knowledge selectively from others — both at the classifier level and the representation level.

- 1 defining mixture priors
- 2 using Gumbel-Softmax trick to learn the mixing weights α_{ti} and β_{ti} , that determine how much task t should rely on task i .

Instead of giving it a fixed prior like a standard Gaussian (which assumes no prior knowledge), for classifiers we let its prior depend on the posteriors of other tasks:

$$p\left(\mathbf{w}_t^{(\eta)} \mid \mathcal{D}_{T \setminus t}\right) = \sum_{i \neq t} \alpha_{ti} q\left(\mathbf{w}_i^{(\eta-1)} \mid \mathcal{D}_i\right)$$

- η : current iteration of training
- $q(\mathbf{w}_i \mid \mathcal{D}_i)$: variational posterior over task i 's classifier from the previous iteration

How Weights are learned?

Remember we claimed that $\mathbf{w}_t$ work on categorical level - we need to choose “relevant tasks”. To model sampling from categorical distribution and to be able to inference through argmax operation. Gumbel-Softmax distribution is used:

$$\alpha_{ti} = \frac{\exp((\log \pi_{ti} + g_{ti}) / \tau)}{\sum_{j \neq t} \exp((\log \pi_{tj} + g_{tj}) / \tau)}$$

where π_{ti} : learnable logits (parameters)
and $g_{ti} \sim \text{Gumbel}(0, 1)$: sampled noise

Priors for Latent Representations

Each sample $\mathbf{x}_{t,n}$ has a latent representation $\mathbf{z}_{t,n}$. As for features representations $\mathbf{z}_{t,n}$ we once again mixes in data from other tasks:

$$p\left(\mathbf{z}_{t,n}^{(\eta)} \mid \mathbf{x}_{t,n}, \mathcal{D}_{T \setminus t}\right) = \sum_{i \neq t} \beta_{ti} q\left(\mathbf{z}_{t,n}^{(\eta-1)} \mid \mathbf{x}_{t,n}, D_i\right),$$

where $q(\mathbf{z}_{t,n} \mid \mathbf{x}_{t,n}, D_i)$ is posterior over representation using data from task i ; and β_{ti} : learned Gumbel-Softmax weights similarly to α_{ti}

Finally, to compute $q(\mathbf{z}_{t,n} \mid \mathbf{x}_{t,n}, D_i)$, we can use cross-attention. The sample $\mathbf{x}_{t,n}$ from the current task acts as a query, and $D_{i,c}$ plays the roles of the key and the value. $D_{i,c}$ is a matrix, with each row containing a sample from class c of the i -th related task and it includes samples from the i -th task, which have the same label as $\mathbf{x}_{t,n}$.

$$f(\mathbf{x}_{t,n}, D_{i,c}) = \text{softmax}\left(\frac{\mathbf{x}_{t,n} D_{i,c}^\top}{\sqrt{d}}\right) D_{i,c}$$

Table 1: Effectiveness of variational Bayesian representations ($\mathbf{z}$) and classifiers ($\mathbf{w}$).

$\mathbf{z}$	$\mathbf{w}$	5%	10%	20%
×	×	50.4 $\pm$ 0.1	59.5 $\pm$ 0.1	65.6 $\pm$ 0.1
✓	×	57.4 $\pm$ 0.0	64.4 $\pm$ 0.1	68.5 $\pm$ 0.1
×	✓	56.8 $\pm$ 0.1	63.7 $\pm$ 0.1	68.5 $\pm$ 0.1
✓	✓	58.3$\pm$0.1	65.0$\pm$0.0	69.2$\pm$0.2

Table 2: Performance comparison of our Gumbel-Softmax priors with other alternative priors.

Priors	5%	10%	20%
Mean	57.5 $\pm$ 0.0	64.4 $\pm$ 0.1	68.5 $\pm$ 0.1
Learnable weighted	57.2 $\pm$ 0.2	64.2 $\pm$ 0.2	68.9 $\pm$ 0.2
Gumbel-Softmax	58.3$\pm$0.1	65.0$\pm$0.0	69.2$\pm$0.2

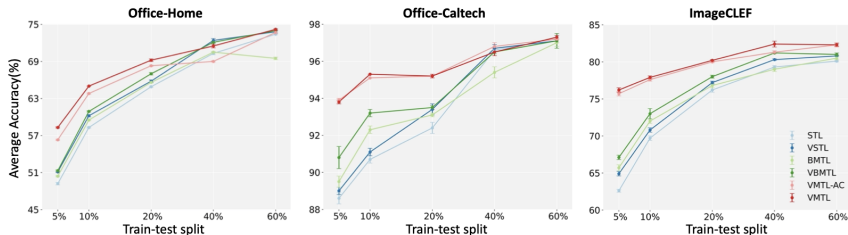


Fig. 2: The performance in terms of average accuracy with a 95% confidence interval under different proportions of training data on *Office-Home*, *Office-Caltech* and *ImageCLEF*. Best viewed in color.

Weights Patterns

Let's see the Gumbel-Softmax mixing weights, visualized as heatmaps between tasks on the Office-Home dataset.

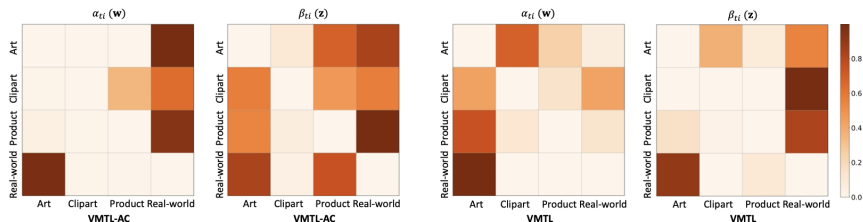


Fig. 3: Tasks show different correlation patterns α_t and β_t for classifiers ($\mathbf{w}$) and representations ($\mathbf{z}$). Similar tasks are more correlated, e.g., Real_World is closer to Art than Clipart.

- Tasks only borrow from a few other related tasks, as Gumbel-Softmax encourages selective sharing.
- There is semantic coherence - as Real-World (R) is closer to Product (P) and Art (A) than Clipart (C) (the last is more stylised).